HW3 Gene Expression

Dylan Renard

Professor Brandon Higgs

Gene Expression and Data Visualization

Johns Hopkins University

November 1st, 2025

Q1) Load the golub data training set in the multtest library. Also load Biobase and annotate libraries, if they are not loaded with the multtest library. Remember that the golub data training set is in the multtest library, so see the help file for information on this data set (2.5 pts)

▼ Code

```
### Q1) Load the Golub training set (robust parse + DT preview) ----
suppressPackageStartupMessages({
  library(Biobase)
  library(DT)
  library(parallel)
  library(limma)
})

# -- file paths (local first, then /mnt/data) --
f_expr  <- if (file.exists("golubTrain.txt")) "golubTrain.txt" else "/mnt/data/golubTrain
f_class <- if (file.exists("golubTrainClass.txt")) "golubTrainClass.txt" else "/mnt/data/

# -- read raw, then clean: many Golub dumps have an ID column and/or a header row --
raw_df <- read.table(f_expr, header = FALSE, sep = "", quote = "", comment.char = "",
                     stringsAsFactors = FALSE, check.names = FALSE)
y_raw  <- scan(f_class, quiet = TRUE)

cat("Raw dim read:", paste(dim(raw_df), collapse = " x "), "\n")
```

```
Raw dim read: 7130 x 39
```

▼ Code

```
cat("Length of labels:", length(y_raw), "\n")
```

```
Length of labels: 38
```

▼ Code

```
# Helper: is a column mostly numeric?
is_mostly_numeric <- function(v) {
  suppressWarnings({
    as_num <- suppressWarnings(as.numeric(v))
    mean(!is.na(as_num)) >= 0.9  # at least 90% numeric
  })
}

# 1) If we have one extra column (e.g., gene IDs), drop the *leftmost* non-numeric column
if (ncol(raw_df) == length(y_raw) + 1) {
  first_col_numeric <- is_mostly_numeric(raw_df[[1]])
  last_col_numeric  <- is_mostly_numeric(raw_df[[ncol(raw_df)]])
  if (!first_col_numeric) {
    cat("Dropping leftmost non-numeric ID column.\n")
    raw_df <- raw_df[-1]  # drop first column
  } else if (!last_col_numeric) {
    cat("Dropping rightmost non-numeric ID column.\n")
    raw_df <- raw_df[-ncol(raw_df)]  # rare, but safe
  }
}
```

```
Dropping leftmost non-numeric ID column.
```

▼ Code

```
# 2) If we still have a mismatch, check for a header row (non-numeric row of sample names
if (nrow(raw_df) == length(y_raw) + 1 && ncol(raw_df) != length(y_raw)) {
  # This case is rare; more common is header row with extra samples, so handle genericall
  cat("Potential header row detected by row count.\n")
}

# Detect a top header row: row 1 mostly non-numeric while others numeric
first_row_mostly_nonnum <- {
  test <- suppressWarnings(as.numeric(raw_df[1, ]))
  mean(is.na(test)) > 0.5
}
second_row_more_numeric <- {
  if (nrow(raw_df) >= 2) {
    test <- suppressWarnings(as.numeric(raw_df[2, ]))
    mean(is.na(test)) < 0.5
  } else FALSE
}

if (first_row_mostly_nonnum && second_row_more_numeric) {
  cat("Dropping top header row.\n")
  raw_df <- raw_df[-1, , drop = FALSE]
}
```

```
# Coerce all remaining to numeric matrix
X_num <- as.matrix(data.frame(lapply(raw_df, function(col) suppressWarnings(as.numeric(co
                              check.names = FALSE))
mode(X_num) <- "numeric"

cat("Post-clean dim:", paste(dim(X_num), collapse = " x "), "\n")
```

```
Post-clean dim: 7130 x 38
```

▼ Code

```
# 3) Auto-orient so COLUMNS are samples and match the label length
if (ncol(X_num) == length(y_raw)) {
  X <- X_num
} else if (nrow(X_num) == length(y_raw)) {
  X <- t(X_num); cat("Transposed matrix so columns = samples.\n")
} else {
  stop(sprintf(
    "Still cannot align data and labels.\n  dim(X) = %d x %d, length(y) = %d\n  One matri
    nrow(X_num), ncol(X_num), length(y_raw)
  ))
}

# 4) Build phenotype and ExpressionSet
pheno <- data.frame(Class = factor(y_raw, levels = c(0,1), labels = c("ALL","AML")))
rownames(pheno) <- paste0("S", seq_len(nrow(pheno)))
colnames(X)     <- rownames(pheno)
rownames(X)     <- paste0("g", seq_len(nrow(X)))

golub_eset <- ExpressionSet(assayData = X, phenoData = new("AnnotatedDataFrame", data = p

# 5) Summaries + interactive head
cat("Final dims (genes x samples):", paste(dim(exprs(golub_eset)), collapse = " x "), "\n
```

```
Final dims (genes x samples): 7130 x 38
```

▼ Code

```
cat("Class distribution (ALL vs AML):\n"); print(table(pheno$Class))
```

```
Class distribution (ALL vs AML):


ALL AML
 27  11
```

▼ Code

```
datatable(
  head(as.data.frame(exprs(golub_eset)), 10),
  options = list(pageLength = 5, scrollX = TRUE),
  caption = "Head of Golub Training Set (genes × samples)"
)
```

Show 5 entries Search:

	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	
g1	1	2	3	4	5	6	7	8	9	10	11	1
g2	-214	-139	-76	-135	-106	-138	-72	-413	5	-88	-165	-6
g3	-153	-73	-49	-114	-125	-85	-144	-260	-127	-105	-155	-9
g4	-58	-1	-307	265	-76	215	238	7	106	42	-71	8
g5	88	283	309	12	168	71	55	-2	268	219	82	2

Head of Golub Training Set (genes × samples)

Showing 1 to 5 of 10 entries Previous 1 2 Next

Q2) Cast the matrix to a data frame and label the gene names as numbers (e.g. “g1”,”g2”,etc). (2.5 pts)

▼ Code

```
### Q2) Cast matrix to data frame and label gene names ----

# Extract the expression matrix from our ExpressionSet
golub_matrix <- exprs(golub_eset)

# Convert to data frame
golub_df <- as.data.frame(golub_matrix)

# Label each row (gene) numerically as g1, g2, ...
rownames(golub_df) <- paste0("g", seq_len(nrow(golub_df)))

# Confirm shape and preview
cat("Dimensions of golub_df:", paste(dim(golub_df), collapse = " x "), "\n")
```

```
Dimensions of golub_df: 7130 x 38
```

▼ Code

```
cat("First 5 gene names:", paste(head(rownames(golub_df), 5), collapse = ", "), "\n")
```

```
First 5 gene names: g1, g2, g3, g4, g5
```

Q3) Get the sample labels (see lecture notes) and set the sample labels to the data frame. (2.5 pts)

▼ Code

```
### Q3) Attach the sample labels to the data frame ----

# Get the class labels from the ExpressionSet phenotype data
sample_labels <- pData(golub_eset)$Class
cat("Class labels (first 10):", paste(head(sample_labels, 10), collapse = ", "), "\n")
```

```
Class labels (first 10): ALL, ALL, ALL, ALL, ALL, ALL, ALL, ALL, ALL, ALL
```

▼ Code

```
golub_df_with_labels <- data.frame(
  Class = sample_labels,
  t(golub_df),                 # genes become columns, samples are rows
  check.names = FALSE
)

# Display interactive table of the first few rows
datatable(
  head(golub_df_with_labels, 10),
  options = list(pageLength = 5, scrollX = TRUE),
  caption = "Head of golub_df_with_labels (samples with class labels)"
)
```

```
Warning in instance$preRenderHook(instance): It seems your data is too big for
client-side DataTables. You may consider server-side processing:
https://rstudio.github.io/DT/server.html
```

Show 5 entries Search:

	Class	g1	g2	g3	g4	g5	g6	g7	g8	g9	g10	
S1	ALL	1	-214	-153	-58	88	-295	-558	199	-176	252	2
S2	ALL	2	-139	-73	-1	283	-264	-400	-330	-168	101	
S3	ALL	3	-76	-49	-307	309	-376	-650	33	-367	206	-2
S4	ALL	4	-135	-114	265	12	-419	-585	158	-253	49	
S5	ALL	5	-106	-125	-76	168	-230	-284	4	-122	70	2

Head of golub_df_with_labels (samples with class labels)

Showing 1 to 5 of 10 entries Previous 1 2 Next

Q4) Use the t-test function in the lecture #7 notes and modify it to “wilcox.test” instead of “t.test”. Change the p.value argument to “$statistic”. Assign the following arguments to the function: (2.5 pts)

> exact=F alternative=“two.sided” correct=T

Run the function on all of the genes in the dataset and >save it as “original.wmw.run”

▼ Code

```
### Q4) Wilcoxon (WMW) test per gene, return W statistic ----

wilcox_func <- function(x, cl) {
  # x: numeric vector for one gene across samples
  x <- as.numeric(x)
  cl <- as.factor(cl)
  stopifnot(length(x) == length(cl))

  g1 <- x[cl == levels(cl)[1]]  # typically "ALL"
  g2 <- x[cl == levels(cl)[2]]  # typically "AML"

  out <- suppressWarnings(wilcox.test(g1, g2, alternative = "two.sided",
                                      exact = FALSE, correct = TRUE))
  unname(out$statistic)  # return W (not p-value)
}

# apply over gene columns (all columns after "Class")
wilcox_stats <- apply(golub_df_with_labels[, -1, drop = FALSE], 2, wilcox_func,
                      cl = golub_df_with_labels$Class)

# keep for later steps
original.wmw.run <- wilcox_stats

cat("Computed Wilcoxon W for", length(wilcox_stats), "genes.\n")
```

```
Computed Wilcoxon W for 7130 genes.
```

▼ Code

```
range(wilcox_stats)
```

```
[1]   0 288
```

Q5) Now write a for loop to iterate 500 times, where in each iteration, the columns of the data frame are shuffled (class labels mixed up), the WMW test is calculated on all of the genes, and the maximum test statistic (W) is saved in a list. (5 pts)

Hints: Use sample() to sample the number of columns Get the maximum test statistic across all genes with max()

▼ Code

```
### Q5 (fast): Parallel permutation test with on-disk caching ----
# Uses mclapply on macOS/Linux. Falls back to serial on Windows.

suppressPackageStartupMessages({ library(parallel) })

# Reproducible parallel RNG
set.seed(410671)
RNGkind("L'Ecuyer-CMRG")

B <- 500

# Cache key includes B, seed, #genes, #samples
n_samples <- nrow(golub_df_with_labels)
n_genes   <- ncol(golub_df_with_labels) - 1

cache_dir  <- "cache"
if (!dir.exists(cache_dir)) dir.create(cache_dir, recursive = TRUE)
cache_file <- file.path(
  cache_dir,
  sprintf("Q5_maxW_B%d_seed%d_%dgenes_%dsamples.rds",
          B, 410671, n_genes, n_samples)
)

if (file.exists(cache_file)) {
  message("Q5-par: Loading cached max_W_list from: ", cache_file)
  max_W_list <- readRDS(cache_file)
} else {
  message("Q5-par: No cache found. Running ", B, " permutations in parallel…")

  # Prepare the permuted label list up front (so each task is independent)
  label_vec <- golub_df_with_labels$Class
  perm_labels <- replicate(B, sample(label_vec, replace = FALSE), simplify = FALSE)

  # Function to compute max W for one permutation
  one_perm_maxW <- function(cl_perm) {
    # wilcox_func defined in Q4
    Wb <- apply(golub_df_with_labels[, -1, drop = FALSE], 2, wilcox_func, cl = cl_perm)
    max(Wb, na.rm = TRUE)
  }
```

```
  # Choose cores (leave 1 free)
  cores <- max(1, detectCores() - 1)

  if (.Platform$OS.type == "windows") {
    # Fallback (Windows doesn't support forking)
    max_W_list <- vapply(perm_labels, one_perm_maxW, numeric(1))
  } else {
    # Parallel on macOS/Linux (your M1 shines here)
    max_W_list <- unlist(mclapply(perm_labels, one_perm_maxW, mc.cores = cores))
  }

  saveRDS(max_W_list, cache_file)
  message("Q5-par: Saved cache to: ", cache_file)
}
```

```
Q5-par: Loading cached max_W_list from:
cache/Q5_maxW_B500_seed410671_7130genes_38samples.rds
```

▼ Code

```
# Summary + threshold
cat("Finished", length(max_W_list), "permutations.\n")
```

```
Finished 500 permutations.
```

▼ Code

```
cat("Null (max W) summary:\n"); print(summary(max_W_list))
```

```
Null (max W) summary:

   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  229.0   247.9   253.5   254.1   260.0   279.0
```

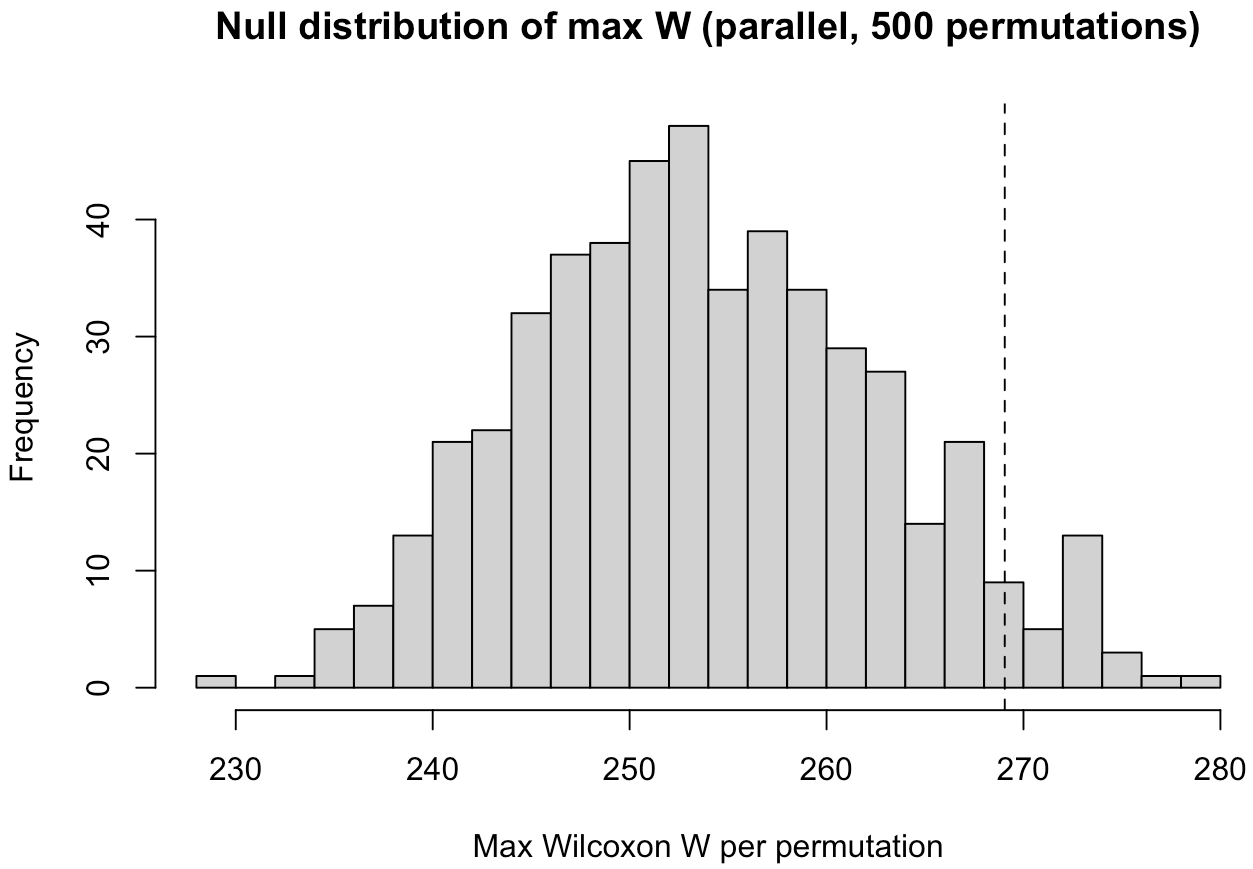
▼ Code

```
W_95 <- as.numeric(quantile(max_W_list, 0.95))
cat("95th percentile of max-W:", W_95, "\n")
```

```
95th percentile of max-W: 269.05
```

▼ Code

```
# Optional quick viz
hist(max_W_list, breaks = 30,
     main = "Null distribution of max W (parallel, 500 permutations)",
     xlab = "Max Wilcoxon W per permutation")
abline(v = W_95, lty = 2)
```



Q6) Once you have the list of maximum test statistics, get the 95% value test statistic. Subset the original.wmw.run list of values with only those that have a higher test statistic than the 95% value that you calculated. Print the gene names and test statistics out. (5 pts)

▼ Code

```
### Q6) Identify significant genes using 95th percentile threshold ----

# Ensure both objects exist
stopifnot(exists("original.wmw.run"), exists("max_W_list"))

# 95th percentile threshold from permutation null
W_95 <- as.numeric(quantile(max_W_list, probs = 0.95))
cat("95th percentile of max-W (null distribution):", W_95, "\n")
```

```
95th percentile of max-W (null distribution): 269.05
```

▼ Code

```
# Subset the original Wilcoxon run: genes exceeding that threshold
sig_idx <- which(original.wmw.run > W_95)
```

```
sig_genes <- names(original.wmw.run)[sig_idx]
sig_stats <- original.wmw.run[sig_idx]

cat("Number of significant genes (W > 95th percentile):", length(sig_genes), "\n")
```

```
Number of significant genes (W > 95th percentile): 22
```

▼ Code

```
# Create a data frame of significant genes and their W statistics
sig_df <- data.frame(
  Gene = sig_genes,
  W_Statistic = sig_stats
)

# Sort in descending order of W
sig_df <- sig_df[order(-sig_df$W_Statistic), ]

# Print top results interactively
if (nrow(sig_df) > 0) {
  datatable(
    sig_df,
    options = list(pageLength = 10, scrollX = TRUE),
    caption = sprintf(
      "Significant Genes (W > %.2f) — Top %d of %d total",
      W_95, min(10, nrow(sig_df)), nrow(sig_df)
    )
  )
} else {
  cat("No genes exceeded the 95th percentile threshold.\n")
}
```

Show 10 entries Search:

	Gene	W
g5773	g5773	288
g2355	g2355	284
g6856	g6856	284
g1145	g1145	280
g1631	g1631	280
g4536	g4536	280
g6282	g6282	280

g2234	g2234	279
g1929	g1929	277
g805	g805	276

Significant Genes (W > 269.05) — Top 10 of 22 total

Showing 1 to 10 of 22 entries Previous 1 2 3 Next

Q7) Now we want to compare these results to those using the empirical Bayes method in the limma package. Load this library and calculate p-values for the same dataset using the eBayes() function. (5 pts)

▼ Code

```
# Prepare design matrix for 2-class comparison
# Intercept + class indicator (AML vs ALL)
y <- factor(golub_df_with_labels$Class, levels = c("ALL", "AML"))
design <- model.matrix(~ y)

# Fit linear model for each gene
fit <- lmFit(t(golub_df_with_labels[, -1, drop = FALSE]), design)

# Apply empirical Bayes moderation
fit <- eBayes(fit)

# Extract p-values for the class coefficient (second column)
limma_pvals <- fit$p.value[, 2]
limma_tstats <- fit$t[, 2]

# Combine results in a tidy data frame
limma_df <- data.frame(
  Gene = names(limma_pvals),
  p_value = limma_pvals,
  t_stat = limma_tstats,
  adj_p_BH = p.adjust(limma_pvals, method = "BH")
)

# Sort by smallest p-value
limma_df <- limma_df[order(limma_df$p_value), ]

# Summary
cat("Number of genes tested:", nrow(limma_df), "\n")
```

```
Number of genes tested: 7130
```

▼ Code

```
cat("Genes with adj p < 0.05:", sum(limma_df$adj_p_BH < 0.05), "\n")
```

```
Genes with adj p < 0.05: 510
```

▼ Code

```
# Interactive table (top 10)
datatable(
  head(limma_df, 10),
  options = list(pageLength = 10, scrollX = TRUE),
  caption = "Top 10 genes from limma eBayes analysis (sorted by p-value)"
)
```

Show 10 entries Search:

	Gene	p_value	t_stat	
g3321	g3321	7.446662759633062e-11	8.994852786375125	4.698269351887363e-7
g4848	g4848	1.31788761623769e-10	8.796047165689856	4.698269351887363e-7
g2021	g2021	3.779861009504781e-10	8.433432970751664	8.98346966592303e-7
g1746	g1746	5.987511855866233e-9	7.506170607059691	0.000089825498758305852
g5040	g5040	6.259115538783908e-9	7.491521676598475	0.000089825498758305852
g1835	g1835	9.925633003908057e-9	7.339661078142896	0.00011794960552977 41
g4197	g4197	3.900036400237173e-8	6.892751675106143	0.00039567980818261 17
g3848	g3848	4.823792477418094e-8	6.823776737207177	0.00039567980818261 17
g462	g462	4.994555783511227e-8	6.812499063305501	0.00039567980818261 17
g2289	g2289	5.714888375835376e-8	6.768849206215549	0.00040747154119706 23

Top 10 genes from limma eBayes analysis (sorted by p-value)

Showing 1 to 10 of 10 entries Previous 1 Next

Q8) Sort the empirical Bayes p-values and acquire the lowest n p-values, where n is defined as the number of significant test statistics that you found in problem 6. Intersect the gene names for your two methods and report how many are in common between the two differential expression methods, when choosing the top n genes from each set. (2.5 pts)

▼ Code

```
### Q8) Compare top-n genes between Wilcoxon and limma eBayes ----

# Ensure both results exist
stopifnot(exists("sig_df"), exists("limma_df"))

# n = number of significant genes from Wilcoxon test (Q6)
n <- nrow(sig_df)
cat("Number of WMW-significant genes (n):", n, "\n")
```

```
Number of WMW-significant genes (n): 22
```

▼ Code

```
if (n > 0) {
  # Get top n genes from limma (lowest p-values)
  top_limma <- head(limma_df$Gene[order(limma_df$p_value)], n)

  # Get top n genes from WMW (highest W statistic)
  top_wmw <- head(sig_df$Gene[order(-sig_df$W_Statistic)], n)

  # Compute intersection
  overlap <- intersect(top_wmw, top_limma)
  overlap_count <- length(overlap)

  cat("Overlap between top", n, "genes (WMW vs limma):", overlap_count, "\n")

  # Optional: data frame for visualization
  overlap_df <- data.frame(
    Gene = overlap,
    WMW_W = original.wmw.run[overlap],
    Limma_p = limma_df$p_value[match(overlap, limma_df$Gene)]
  )

  datatable(
    overlap_df,
    options = list(pageLength = 10, scrollX = TRUE),
    caption = sprintf(
      "Overlap of top %d genes between WMW and limma (count = %d)",
```

```
      n, overlap_count
    )
  )
} else {
  cat("No significant genes in WMW results (n=0), cannot compute overlap.\n")
}
```

```
Overlap between top 22 genes (WMW vs limma): 0
```

Show 10 entries Search:

Gene	WMW_W
No data available in table	

Overlap of top 22 genes between WMW and limma (count = 0)

Showing 0 to 0 of 0 entries Previous Next

Q9) Finally, compare the results from a Student's t-test with the empirical Bayes method. To do this, first calculate a two sample (two-tailed) Student's t-test on all genes. Make sure that you are running a Student's t-test and not a Welch's t-test. Then extract only those genes with a p-value less than 0.01 from this test. Plot the gene p-values<0.01 for the Student's t-test vs. the same genes in the empirical Bayes method. Make sure to label the axes and title appropriately. (7.5 pts)

▼ Code

```
### Q9) Compare Student's t-test with empirical Bayes (limma) ----

# Sanity: confirm data exists
stopifnot(exists("golub_df_with_labels"), exists("limma_df"))

# Prepare class indices
idx_ALL <- which(golub_df_with_labels$Class == "ALL")
idx_AML <- which(golub_df_with_labels$Class == "AML")

# Perform Student's (equal-variance) t-test for each gene
student_t_pvals <- apply(golub_df_with_labels[, -1, drop = FALSE], 2, function(x) {
  t.test(x[idx_ALL], x[idx_AML], var.equal = TRUE)$p.value
})

# Number of genes passing p < 0.01
sig_t_idx <- which(student_t_pvals < 0.01)
cat("Genes with Student's t-test p < 0.01:", length(sig_t_idx), "\n")
```

```
Genes with Student's t-test p < 0.01: 800
```

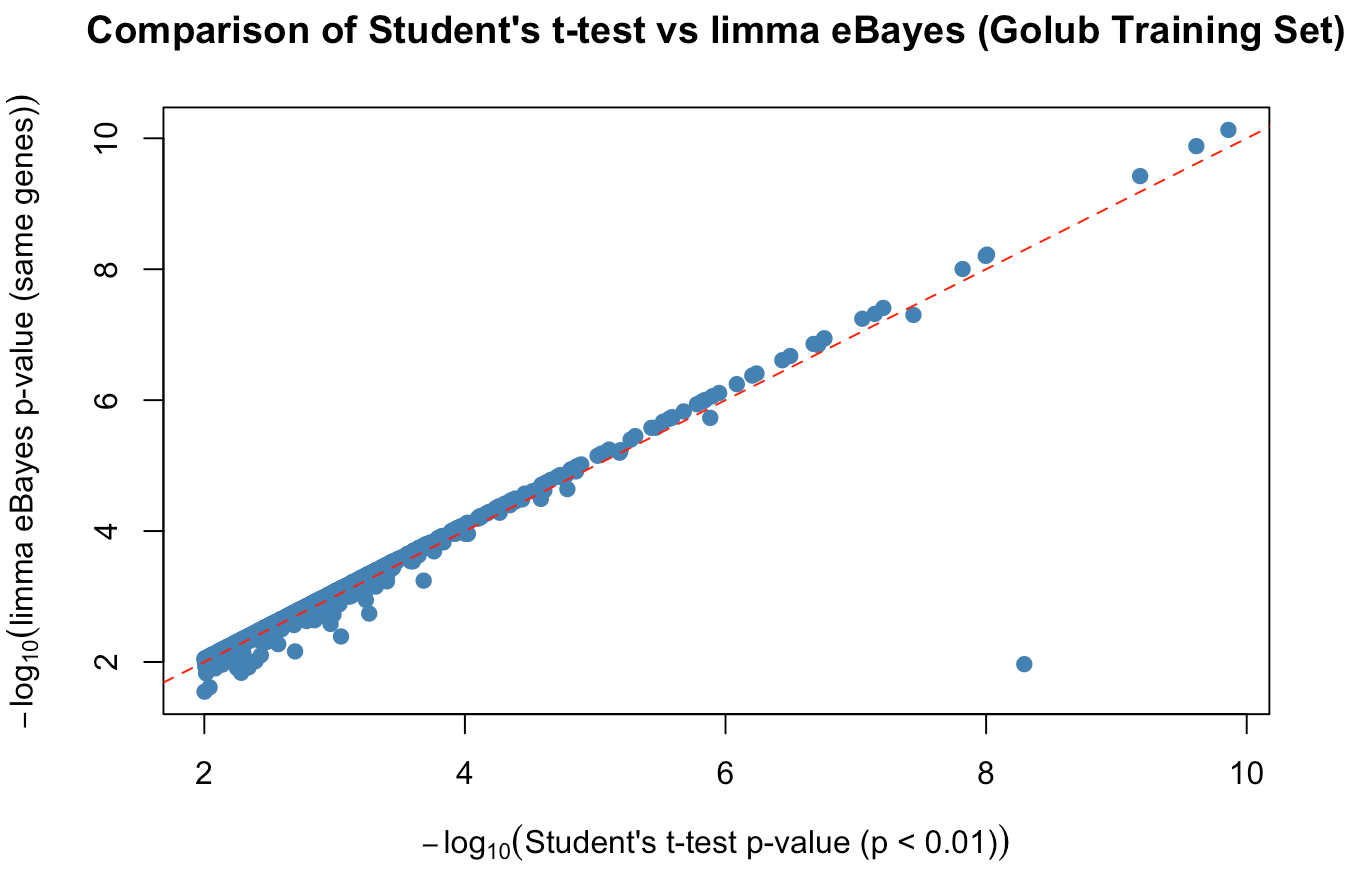
▼ Code

```r
if (length(sig_t_idx) > 0) {
  # Extract gene names and matching limma p-values
  sig_genes_t <- names(student_t_pvals)[sig_t_idx]
  p_t <- student_t_pvals[sig_genes_t]
  p_limma <- limma_df$p_value[match(sig_genes_t, limma_df$Gene)]

  # Combine into a dataframe for plotting
  compare_df <- data.frame(
    Gene = sig_genes_t,
    Student_t_p = p_t,
    Limma_p = p_limma
  )

  # Scatter plot: raw p-values (both on log10 scale for clarity)
  plot(
    -log10(compare_df$Student_t_p),
    -log10(compare_df$Limma_p),
    pch = 19, col = "steelblue",
    xlab = expression(-log[10](paste("Student's t-test p-value (p < 0.01)"))),
    ylab = expression(-log[10](paste("limma eBayes p-value (same genes)"))),
    main = "Comparison of Student's t-test vs limma eBayes (Golub Training Set)"
  )
  abline(0, 1, lty = 2, col = "red")

  # Optional DT preview of first 10 genes
  datatable(
    head(compare_df[order(compare_df$Student_t_p), ], 10),
    options = list(pageLength = 10, scrollX = TRUE),
    caption = "Top 10 genes (Student's t-test p < 0.01) with corresponding limma p-values
  )
} else {
  cat("No genes found with Student's t-test p < 0.01.\n")
}
```



Show 10 entries Search:

	Gene	Student_t_p	
g3321	g3321	1.382385657332042e-10	7.446662759633062e-11
g4848	g4848	2.435528473277847e-10	1.31788761623769e-10
g2021	g2021	6.577977412771993e-10	3.779861009504781e-10
g1	g1	5.090377997318544e-9	0.01079839638305398
g1746	g1746	9.846897254435025e-9	5.987511855866233e-9
g5040	g5040	1.005968567021927e-8	6.259115538783908e-9
g1835	g1835	1.514532216430054e-8	9.925633003908057e-9
g462	g462	3.610124227026674e-8	4.994555783511227e-8
g4197	g4197	6.156940644228055e-8	3.900036400237173e-8
g3848	g3848	7.157106018903862e-8	4.823792477418094e-8

Top 10 genes (Student's t-test p < 0.01) with corresponding limma p-values

Showing 1 to 10 of 10 entries

Previous 1 Next